

마왕 채널 (MAOU CHANNEL) — AI 활용 기술 문서

최종 수정 2026-08-10

1. 문서 개요

1.1 목적

본 문서는 마왕 채널(MAOU CHANNEL) 개발 과정에서 생성형 AI를 어떤 구조로, 어떤 지시(프롬프트)로, 어느 범위까지 활용했는지를 기술적으로 설명한다. 또한 프로젝트에 포함된 외부 에셋과 오픈소스의 출처·라이선스를 명시한다.

1.2 프로젝트 개요

항목	내용
제목	마왕 채널 (MAOU CHANNEL)
장르	오토배틀 × 육성 × 리듬 (라이브 스트리밍 시뮬레이션)
플랫폼	웹 브라우저 (클라이언트 전용, 서버 없음)
기술 스택	TypeScript 7 · React 18 · Phaser 3 · Zustand · Vite
배포	GitHub Pages (main push 시 GitHub Actions 자동 배포)
개발 규모	2인 / 2주
저장소	https://github.com/tnghgks/Project_MAOU

1.3 핵심 요약 — 어디에 AI를 썼고, 어디에 쓰지 않았나

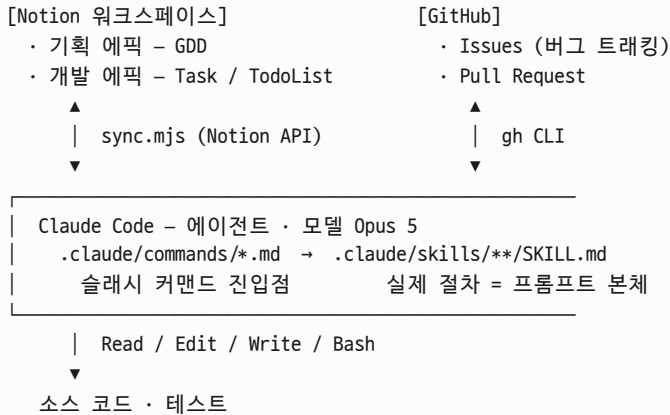
구분	AI 활용 여부	비고
게임 런타임(플레이 중 AI 추론)	사용 안 함	게임은 외부 AI API를 호출하지 않는다. 시청자 채팅·닉네임·몬스터 AI는 전부 결정론적 규칙과 난수 기반 코드다.
코드 구현 · 리팩터링	사용	Claude Code (Opus 5)
기획서(GDD) ↔ 코드 동기화	사용	Claude Code (Opus 5) + Notion API 커스텀 스킬
이슈 분석 → 수정 → PR	사용	Claude Code (Opus 5) + GitHub CLI 커스텀 스킬
캐릭터 · 몬스터 스프라이트	사용	PixelLab.ai (image to pixel art + character) + Claude 작성 캐릭터 디스크립션 프롬프트

구분	AI 활용 여부	비고
컷씬 영상	사용	Higgsfield(마스터 시트) → GPT Image 2(스토리보드) → Higgsfield MCP · Seedance 2.0(영상)
BGM	사용	Suno AI로 직접 제작
후원 효과음	사용	ElevenLabs로 직접 제작
그 외 효과음	사용 안 함	상업적 이용이 가능한 외부 SFX 팩 (JDSherbert)

중요 — 플레이어에게 전달되는 빌드 결과물(dist/)에는 AI 모델도, AI API 키도, AI 호출 코드도 포함되지 않는다. 소스 전체에서 외부로 나가는 네트워크 요청은 스프라이트 아틀라스 JSON을 읽는 `fetch` 한 곳 (src/ui/useSpriteThumb.ts:60)과 컷씬 영상 로드뿐이다.

2. AI 활용 전체 구조

개발 시점 (Development Time)



— 에셋 생성 —

PixelLab.ai (레퍼런스 + 캐릭터 디스크립션 프롬프트)
↳ assets/character/<이름>/
 ↳ scripts/pack.js (의존성 0 패커)
 ↳ public/assets/character/*.png + *.json

Higgsfield → GPT Image 2 → Seedance 2.0 (Higgsfield MCP)
 마스터 시트 스토리보드 컷썬 영상
 ↳ 오브젝트 스토리지 버킷 /cutscenes/*.mp4

Suno AI → BGM ElevenLabs → 후원 효과음

npm run build (Vite)

실행 시점 (Runtime)

브라우저: React(UI · 썬 디렉터) + Phaser 3(캔버스) + Zustand(상태)
※ AI 추론 없음 - 전부 사전 생성된 정적 에셋과 결정론적 로직

핵심 설계 의도는 "AI는 개발 파이프라인 안에서만 동작하고, 산출물은 전부 사람이 리뷰한 정적 자산으로 고정된다" 이다. 플레이 중 품질이 흔들릴 여지(모델 응답 편차, API 장애, 지연, 비용)를 런타임에서 완전히 제거했다.

3. 코드 개발에서의 AI 활용 구조

3.1 에이전트 하네스 — 커맨드/스킬 2단 구조

코드 작업에 사용한 AI는 **Claude Code(모델: Opus 5)**이며, AI에게 주는 지시는 대화 중 즉흥적으로 던지지 않고 저장소에 파일로 버전 관리한다.

```
.claude/
├── commands/                # 슬래시 커맨드 진입점 (얇은 포인터)
│   ├── development-sync.md
│   ├── gdd-pull-sync.md
│   ├── gdd-push-sync.md
│   ├── fix-github-issue.md
│   └── github-issue-pr.md
├── skills/                  # 실제 절차 = 프롬프트 본체
│   ├── fix-github-issue/SKILL.md
│   ├── github-issue-pr/SKILL.md
│   └── notion/
│       ├── development-sync/SKILL.md
│       ├── gdd-pull-sync/SKILL.md
│       ├── gdd-push-sync/SKILL.md
│       └── gdd-shared/      # 두 스킬이 공유하는 상태
│           ├── .state.json  # 버전·마지막 동기화 시각·커밋·이력
│           └── .snapshot.txt # 마지막 동기화 시점의 GDD 전문
└── scripts/sync.mjs         # Notion API CLI (판단 없음, I/O만)
```

2단 구조를 택한 이유: 커맨드 파일은 "스킬 파일을 읽고 그대로 따르라"는 포인터만 담는다. 절차가 바뀔 때 고칠 곳이 스킬 파일 한 곳으로 고정되고, AI가 오래된 요약본을 따라가는 사고를 막는다. 실제 커맨드 파일 전문:

```
`./.claude/skills/notion/gdd-pull-sync/SKILL.md`를 Read 도구로 읽고, 그 안에 적힌 0~10단계를
순서대로 그대로 실행한다. 이 파일은 진입점일 뿐이고 실제 절차는 SKILL.md 쪽이 최신 기준이니
내용을 요약해서 옮겨적지 말고 반드시 그 파일을 직접 읽어서 따른다.
```

3.2 판단과 실행의 분리

sync.mjs (397줄)는 Notion API와만 통신하고 **어떤 판단도 하지 않는다**. 스크립트 헤더에 이 원칙이 명시돼 있다.

```
// Notion read/write CLI for the notion-task-sync skill.
// All matching/analysis judgment lives in the skill instructions (Claude decides what to
// call this script with) - this file only talks to the Notion API.
```

제공 명령: `list` · `create-task` · `create-todo` · `update-todo` · `doc-blocks` · `page-meta` · `update-text-block` · `update-table-row` · `append-block` · `append-table-row` · `create-table` · `delete-block`

이렇게 나눈 결과:

- **재현성** — 같은 명령은 항상 같은 결과. API 호출 로직이 매번 새로 생성되지 않는다.
- **검증 가능성** — AI가 "무엇을 했는지"가 명령어 로그로 남는다.
- **비용** — 세 스킬이 하나의 스크립트를 공유한다(스킬마다 스크립트를 새로 만들지 않는다는 지시가 SKILL.md에 명문화됨).

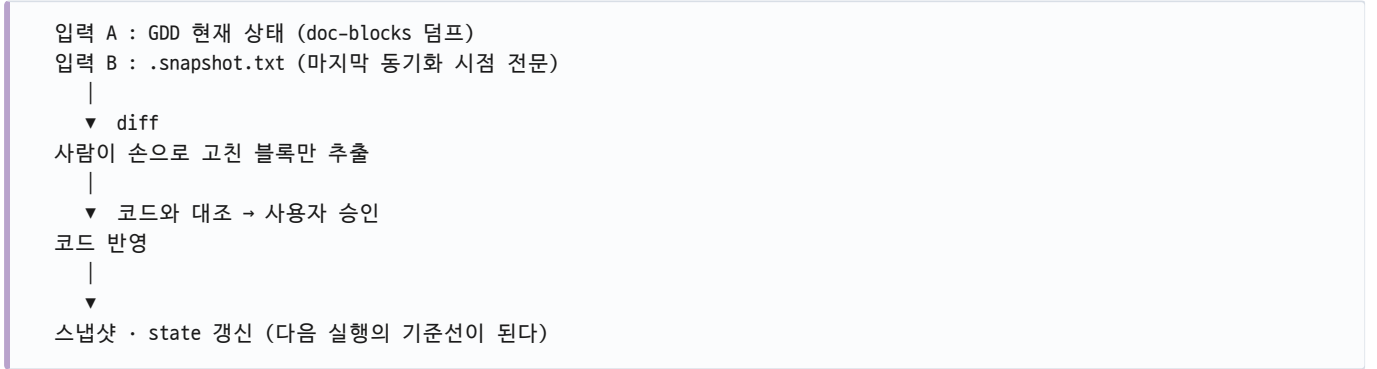
3.3 GDD ↔ 코드 양방향 동기화

2주 개발에서 가장 자주 깨지는 것이 "기획서와 코드의 불일치"다. 이를 **방향이 명시된 세 개의 스킬**로 나눠 해결했다.

스킬	방향	기준(Source of Truth)
/gdd-pull-sync	노션 GDD → 코드	GDD — 기획이 바뀌면 코드를 맞춘다
/gdd-push-sync	코드 → 노션 GDD	코드 — 밸런스를 코드에서 먼저 만졌으면 문서를 맞춘다

스킬	방향	기준(Source of Truth)
<code>/development-sync</code>	코드 → 노션 개발 태스크	코드 — 구현 상태를 Task/ToDoList 진행률에 반영

스냅샷 diff 기법. 사람은 GDD를 고치면서 변경 이력을 남기지 않는다. 그래서 동기화가 끝날 때마다 GDD 전문을 `.snapshot.txt`에 저장해두고, 다음 실행 때 새 덤프와 `diff`를 떼서 "사람이 손으로 고친 블록"만 정확히 골라낸다. 자동화가 고친 부분은 스냅샷에 이미 반영돼 있어 `diff`에 걸리지 않는다.



충돌 방지. `/gdd-push-sync`는 실행 전에 `.state.json`의 `lastGddEditedTime`과 노션의 현재 `lastEditedTime`을 비교한다. 다르면 "사람이 방금 고친 내용을 코드 기준으로 덮어쓸 위험"이 있다고 보고 먼저 **pull-sync**를 돌릴지 사용자에게 확인받는다.

3.4 이슈 → 수정 → PR 파이프라인

`/fix-github-issue`와 `/github-issue-pr`를 **일부러 분리**했다. 조사·수정과 커밋·푸시 사이에 사람의 확인이 반드시 들어가게 하기 위해서다. 전자의 마지막 단계는 이렇게 끝난다.

```

8. **Hand off.** Tell the user the fix is ready and that the `github-issue-pr`
   skill will branch, commit, push, and open the PR referencing this issue.
   Do not commit or push yourself.
  
```

3.5 안전장치 (Guardrails)

AI가 되돌리기 어려운 작업을 하기 전에 멈추도록, 모든 스킬에 아래 게이트를 심었다.

게이트	내용	위치
사전 조건 검사	<code>NOTION_TOKEN</code> 없으면 코드 분석도 시작하지 말고 즉시 중단	각 SKILL.md 0단계
쓰기 전 승인	노션 쓰기·코드 변경 전 계획을 사람에게 보여주고 승인 획득. 일부만 승인되면 그것만 진행	pull-sync 5, push-sync 4, development-sync 3
덮어쓰기 방지	편집 시각 북마크 비교 후 충돌 가능성 경고	push-sync 0단계
이력 보존	변경 이력 표의 기존 행은 절대 수정/삭제 금지 , 추가만	push-sync 8단계
히스토리 보호	<code>force-push</code> · <code>amend</code> 금지	github-issue-pr Notes

게이트	내용	위치
추측 금지	"추측하지 말고 실제로 파일을 읽어서 확인한 것만 반영한다"	development-sync 3단계
비밀정보	토큰은 <code>.env</code> (gitignore)에만. 배포는 GitHub Actions repo variable/secret 주입	<code>.env.example</code> , <code>deploy.yml</code>

3.6 AI 협업을 전제로 한 코드 규약

AI가 잘못 건드리기 쉬운 지점을 코드 자체에 표시해두는 규약을 운영했다.

ponytail: 주식 (49곳) — "여기가 밸런스 조절 손잡이(knob)이며, 구조를 바꾸지 말고 이 값만 만져라"는 표식.

```
export const WAVE_INTERVAL = 15; // 자동 투입 간격(초). ponytail: 방송 템포 knob
export const DASH_CD = 2.5; // ponytail: 회피 난이도 knob
export const JACKPOT_CHANCE = 0.08; // ponytail: 대박 후원(=리액션 이벤트) 확률·배율 knob
```

의도적 설계를 "고치지 못하게" 문서화 — 예를 들어 `HeroEntity` (씬 로컬, 매 프레임 변이)와 `HeroStats` (스토어)를 이중으로 두는 구조는 React 리렌더를 피하려는 의도적 선택이므로 통합하지 않는다.

AI가 검증할 수 있는 구조 — 전투 판정 로직을 Phaser에서 분리해 순수 함수로 두었다 (`src/game/battleSim.ts`, `src/formulas.ts`). 덕분에 브라우저 없이 `npm test` (15개 테스트 스크립트)로 AI가 스스로 변경을 검증할 수 있다.

```
// "엔티티용 formulas.ts" - 전투 시뮬 결정 로직을 Phaser 없이 모은다.
// 헬퍼는 plain 시뮬 필드(x/y/hp/타이머)만 변이하고, Phaser가 필요한 것은 intent로 반환한다.
// 덕분에 브라우저 없이 테스트된다.
```

4. AI 대상 주요 프롬프트 및 지시 사항

4.1 프롬프트 작성 원칙

본 프로젝트에서 AI 지시문을 작성할 때 지킨 규칙:

- 절차를 번호로 고정한다 — "0~10단계를 순서대로 그대로 실행한다". 자유 재량 폭을 좁혀 실행 편차를 줄인다.
- 범위 밖을 명시적으로 배제한다 — 각 스킬에 "이건 이 스킬의 범위가 아니다"를 반드시 적는다.
- 판단 기준을 정량화한다 — "동작하는 코드가 있으면 완료, 일부만 구현됐으면 진행 중, 없으면 시작 전".
- 중단 조건을 앞에 둔다 — 사전 조건 실패 시 아무 작업도 하지 않고 멈춘다.
- 출력 포맷을 예시로 준다 — 변경 이력 표의 셀 포맷까지 예시로 고정.
- 스킬 간 관계를 링크로 표현한다 — `[[gdd-push-sync]]` 형태로 상호 참조.

4.2 스킬 지시문 발췌 (원문)

▶ 범위 경계 지정 — 방향을 명시해 반대 방향 작업을 차단

- 코드에 있는데 GDD 수치/로직과 다르면 → ****GDD 갱신 대상****
- 코드에 새로 생긴 시스템/기능인데 GDD에 아예 섹션이 없으면 → ****GDD에 새 블록 추가 대상****
- GDD에 있는데 코드에 아직 없는 건 이 스킬의 범위가 아니다 (그건 `gdd-pull-sync`가 코드로 옮기는 역할). 여기서는 코드 → GDD 방향만 본다.
- 이미 code = GDD로 일치하는 부분은 건드리지 않는다.

▶ 승인 게이트 — 게임 로직 변경은 문서 변경보다 더 신중하게

찾은 차이점마다 "GDD 내용 / 현재 코드 상태 / 제안하는 코드 변경"을 정리해서 사용자에게 보여주고 ****반드시 승인을 받는다****. 게임 로직·밸런스에 실제 영향을 주는 변경이므로, 노션 쓰기 작업보다 더 신중하게 확인받는다. 일부만 승인되면 승인된 것만 진행한다.

▶ 검증 지시 — 타입 체크 통과를 "완료"로 인정하지 않음

7. ****Verify.**** Run relevant tests/build/lint if the project has them. For UI/gameplay changes, prefer actually running the change over claiming success from a type-check alone.

▶ 기존 패턴 준수 — 새 추상화 도입 금지

6. ****Implement the fix**** with Edit/Write, following this repo's existing patterns and conventions (check CLAUDE.md and neighboring code – don't introduce new abstractions or unrelated cleanup).

▶ 출력 포맷 고정 — 변경 이력 표 셀 포맷까지 지정

`수정사항` 셀 안에서 항목마다 ****상위 불릿****(`• [섹션번호]`) 한 줄 + 그 아래 들여쓴 하위 불릿 (`◦ 내용`) 한 줄**로 2줄씩 묶어 나열한다. 하위 불릿 내용은 ****개조식****(명사형으로 끝내기, "~했다/~됨" 같은 서술형 어미 지양, 짧게)으로 쓴다. 예:

- [3-2. 긴장도]
 - 위험도 공식에 벼랑끝 보너스(+0.15) 문서화

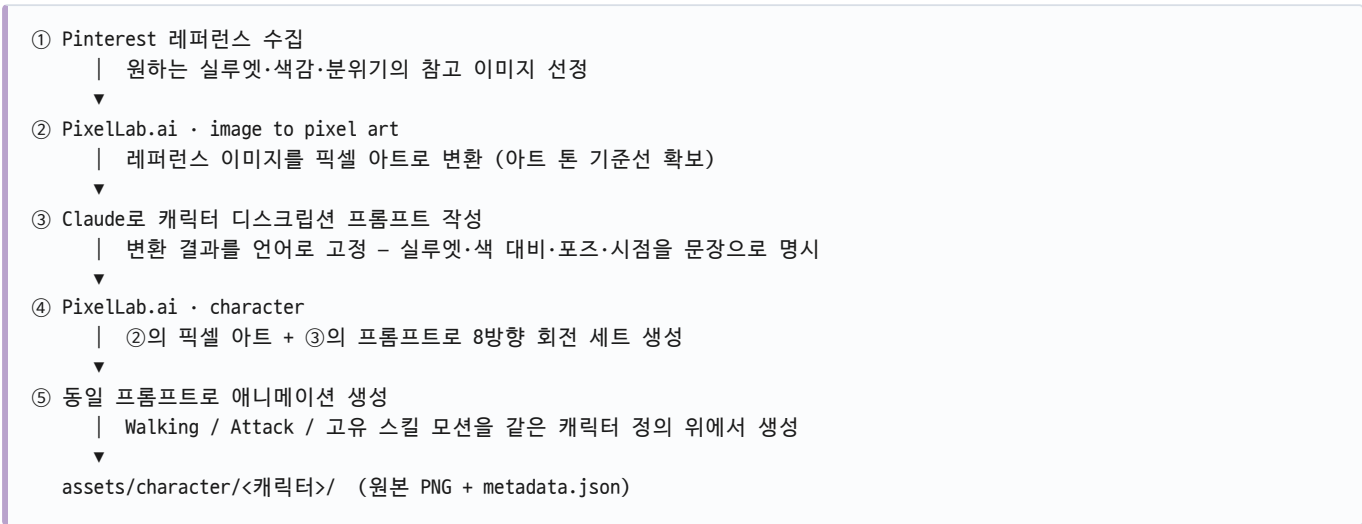
▶ 실패 시 행동 규정 — 추측으로 메우지 않기

- If `gh issue view` fails (auth, not found, wrong repo), surface the raw error rather than guessing at the issue content.

4.3 캐릭터 · 몬스터 스프라이트 생성 프롬프트

사용 서비스: [PixelLab.ai](#) — `image to pixel art` 와 `character` 두 기능을 함께 사용했다. 텍스트 프롬프트로부터 **8 방향 회전 이미지 + 액션별 애니메이션 프레임**을 생성하며, 생성 파라미터와 프롬프트는 `assets/character/<캐릭터>/metadata.json` 에 그대로 보존된다.

4.3.1 워크플로우



이 순서를 택한 이유 — ②에서 얻은 픽셀 아트가 "보이는 기준"이고, ③의 Claude 디스크립션이 "말로 된 기준"이다. 두 기준을 함께 놓아야 ④·⑤의 8방향 회전과 애니메이션에서 캐릭터 정체성이 흔들리지 않는다. 애니메이션을 별도 프롬프트로 다시 쓰지 않고 **같은 디스크립션을 재사용**한 것도 같은 이유다.

4.3.2 실제 사용 예 — 2화 보스 「베르하르트(Verhart)」

③단계에서 Claude가 작성한 캐릭터 디스크립션 프롬프트 원문:

```
towering black knight commander in heavy dark violet plate armor,
wide angular spiked pauldrons, horned helmet with narrow visor
slit and two glowing red eyes, crimson trim lines on chest and
belt, long tattered dark red cape, holding a large two-handed
greatsword upright in front with both hands, blade tip down,
bright rim light on shoulder and helmet edges, readable silhouette
```

파라미터	값	의미
size	176 × 176	스프라이트 해상도
template_id	mannequin	포즈/골격 템플릿
directions	8	8방향 회전 세트 (south ~ south-west)
view	high top-down	게임 카메라 각도와 일치시키기 위한 시점
생성 애니메이션	Walking (9프레임 × 3방향), Attack (9프레임 × 3방향), Whirlwind (16프레임)	액션별 프레임 시퀀스

4.3.3 프롬프트 작성 전략

게임 화면에서 실제로 읽히는 요소에 집중했다.

1. 실루엣 우선 — readable silhouette , towering , wide angular spiked pauldrons . 작은 픽셀 크기에서 형태만으로 식별되도록 유도.
2. 색 대비 명시 — dark violet 갑옷 + crimson trim + glowing red eyes . 어두운 아레나 배경에서 분리되게 하는 장치.

3. 가독성 보조광 지정 — `bright rim light on shoulder and helmet edges`. 배경과 캐릭터의 경계를 강제로 살린다.
4. 포즈 고정 — `holding a greatsword upright in front with both hands, blade tip down`. 8방향 회전을 생성할 때 방향별로 자세가 흔들리는 것을 막는다.
5. 시점 일치 — `view: high top-down` 을 게임 카메라와 맞춰, 생성물을 그대로 쓸 수 있게 한다.

4.4 컷씬 영상 생성 파이프라인

프롤로그·화별 도입·보스 등장 컷씬은 **3단계 도구 체인**으로 제작했다. 각 단계의 산출물이 다음 단계의 입력이 되며, 뒤로 갈수록 자유도를 좁혀 일관성을 확보하는 구조다.

```
① Higgsfield - 캐릭터 · 배경 마스터 시트 생성
  | 등장인물과 무대의 "정답 이미지"를 먼저 고정
  ▼
② GPT Image 2 - 스토리보드 생성
  | 마스터 시트를 참조로 컷 단위 구도·연출 확정
  ▼
③ Higgsfield MCP → Seedance 2.0 - 영상 생성
  | 스토리보드 컷을 영상으로 전개
  ▼
오브젝트 스토리지 버킷 /cutscenes/*.mp4
  | VITE_BUCKET_BASE_URL 로 주입
  ▼
src/data/cutscenes.ts (썸 id ↔ 영상 src 매핑)
```

단계	도구	산출물	역할
①	Higgsfield	캐릭터 · 배경 마스터 시트	컷마다 인물/배경이 달라지는 것을 막는 기준 이미지
②	GPT Image 2	스토리보드	컷 구성 · 카메라 · 연출을 이미지로 확정
③	Seedance 2.0 (Higgsfield MCP 경유)	컷씬 <code>.mp4</code>	스토리보드를 영상으로 전개

마스터 시트를 먼저 만든 이유 — 영상 생성 모델은 호출마다 인물 디테일이 흔들린다. ①에서 캐릭터·배경을 고정해두고 ②·③이 그것을 참조하게 만들어, 여러 컷씬 사이에서 동일 인물로 읽히게 했다.

코드 쪽 연결 구조 — 영상은 저장소에 커밋하지 않고 외부 버킷에 두고 URL만 참조한다 (`src/data/cutscenes.ts`). 버킷 URL이 없어도 게임은 진행되며, 대사 카드로 자동 대체된다.

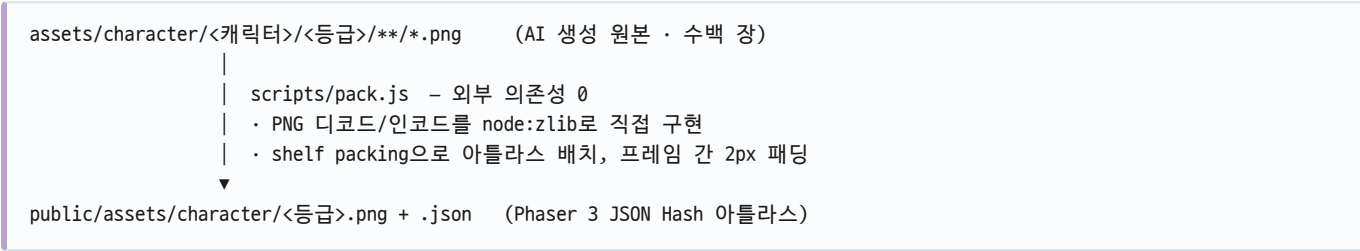
```
const baseUrl = import.meta.env.VITE_BUCKET_BASE_URL || '';

intro: {
  title: '프롤로그',
  lines: ['마왕성의 재정은 파탄났다.', '남은 수익 모델은 단 하나 - 방송.', ...],
  tone: 'dark',
  src: `${baseUrl}/cutscenes/intro.mp4`,
},
```

배포 빌드에서는 GitHub Actions repo variable(`vars.VITE_BUCKET_BASE_URL`)로 주입되고, 로컬 개발에서는 `.env` 가 그 값을 제공한다. 저장소에는 URL이 하드코딩되지 않는다.

4.5 생성 에셋 파이프라인 (스프라이트 → 아틀라스)

AI가 뱀의 낱장 PNG를 게임에서 쓸 수 있는 형태로 바꾸는 단계는 자체 제작 스크립트 (scripts/pack.js, npm run assets)가 담당한다.



패킹 정책도 코드에 근거와 함께 고정해두었다.

```
// 회전은 하지 않는다 - flipX와 섞이면 방향 처리가 꼬인다. 트림도 하지 않는다 - 프레임마다
// origin이 달라져 걷는 동안 인물이 뻐다.

// 입력은 도구가 뱀의 것이라 신뢰 경계다 - 조용히 깨지느니 여기서 멈춘다
if (bitDepth !== 8 || (colorType !== 6 && colorType !== 2) || interlace !== 0)
    throw new Error(`${path}: 지원 안 하는 PNG ...`);
```

즉 AI 산출물을 "신뢰할 수 없는 입력"으로 취급하고 파이프라인 경계에서 검증한다.

5. AI가 관여하지 않은 영역 (설계 판단)

의도적으로 AI를 쓰지 않은 부분을 밝힌다.

영역	구현 방식	AI를 쓰지 않은 이유
시청자 채팅	위험도 구간별 대사 풀에서 샘플링 (src/data/chat.ts)	응답 지연이 전투 템포를 깬다. 한국어 스트리밍 채팅 특유의 톤은 직접 쓰는 편이 정확하다.
시청자 닉네임	[수식어][정체][꼬리] 조합 생성기, 약 2천 조합 (src/data/nicknames.ts)	결정론적 생성으로 충분하고 비용이 0이다.
몬스터 · 보스 AI	순수 함수 상태 머신 (src/game/battleSim.ts)	밸런스 조정과 테스트 재현성이 필수.
밸런스 수식	명시적 공식 + 단위 테스트 (src/formulas.ts)	수치가 예측 가능해야 튜닝이 가능하다.
게임 기획 · 세계관	사람이 작성 (노션 GDD)	프로젝트의 정체성 영역.

6. 개발 프로세스 실측

항목	값
코드 작업 AI	Claude Code · 모델 Opus 5
총 커밋 수	177
PR 수	43 (전량 <code>main</code> 대상, 리뷰 후 머지)
테스트 스크립트	15개 (<code>npm test</code> 로 체이닝)
AI 지시문 파일	커맨드 5 + 스킴 5
ponytail: knob 주석	49곳
GDD 동기화 버전 이력	v0.8까지 진행 (커밋 <code>a2097ae</code> 참조)

7. 외부 에셋 / 오픈소스 출처

아래 내용의 단일 출처(single source of truth) 는 `src/data/credits.ts`이며, 라이선스 전문은 배포물에 함께 실리는 `public/THIRD-PARTY-NOTICES.md`에 있다. 게임 내 타이틀 → 제작자 화면이 같은 데이터를 읽어 표시한다.

7.1 엔진 · 라이브러리

이름	저작자	라이선스	출처
Phaser 3	Phaser Studio Inc.	MIT	https://github.com/phaserjs/phaser
React / React DOM	Meta Platforms, Inc. and affiliates	MIT	https://github.com/facebook/react
Zustand	Poimandres	MIT	https://github.com/pmndrs/zustand
Vite	VoidZero Inc. and Vite contributors	MIT	https://github.com/vitejs/vite
TypeScript	Microsoft	Apache-2.0	https://github.com/microsoft/TypeScript
Prettier	Prettier contributors	MIT	https://github.com/prettier/prettier
gh-pages	Tim Schaub	MIT	https://github.com/tschaub/gh-pages

빌드 결과물에 실제로 포함되는 것은 *Phaser 3 · React · Zustand*이며, 나머지는 개발 도구다.

7.2 폰트

이름	저작자	라이선스	출처
Galmuri (갈무리)	quiple	SIL Open Font License 1.1	https://github.com/quiple/galmuri

OFL 1.1은 재배포 시 라이선스 사본 동봉과 저작권 표시를 요구하며, 폰트 단독 판매를 금지한다. 전문: <https://openfontlicense.org>

7.3 아트 · 사운드

7.3.1 AI로 직접 제작한 산출물

외부에서 가져온 것이 아니라 본 프로젝트가 AI 도구로 직접 생성한 자산이다.

대상	경로	생성 도구	프롬프트·파라미터 보존 위치
캐릭터 · 몬스터 스프라이트 (8방향 회전 + 애니메이션)	public/assets/character/	PixelLab.ai — image to pixel art + character 캐릭터 디스크립션 프롬프트는 Claude 로 작성	assets/character/<캐릭터>/metadata.json (4.3)
컷씬 영상	외부 버킷 /cutscenes/*.mp4	Higgsfield (마스터 시트) → GPT Image 2 (스토리보드) → Seedance 2.0 (Higgsfield MCP 경유)	4.4
BGM	public/assets/sounds/bgm/	Suno AI	—
후원 효과음 (small · middle · big)	public/assets/sounds/sfx/*_donation.mp3	ElevenLabs (무료 플랜)	—

레퍼런스 수집에 사용한 Pinterest 이미지는 **아트 방향 참고용으로만 사용**했으며, 최종 산출물은 PixelLab.ai에서 새로 생성된 픽셀 아트다. 원본 레퍼런스는 배포물에 포함되지 않는다.

7.3.2 외부 에셋

대상	경로	저작자	라이선스
Pixel Game Essentials SFX Pack	public/assets/sounds/sfx/JDSherbert - Pixel Game Essentials SFX Pack - *.ogg	JDSherbert	상업적 이용 가능 (확인 완료)

효과음 팩은 파일명에 팩 이름과 저작자가 그대로 남아 있어 출처 추적이 가능하도록 **리네임하지 않았다**.

고지 정리 기준 (THIRD-PARTY-NOTICES.md에 명시된 규칙):

- **자체 제작 · AI 생성** — 사용한 도구와 저작자를 표기한다.
- **무료/유료 에셋 팩** — 팩 이름 · 제작자 · 라이선스 · 배포 페이지 URL을 그대로 옮겨 적는다. CC BY 계열은 **저작자 표시가 의무**이므로 문서와 게임 내 화면 양쪽에 반드시 노출한다.

7.4 라이선스 고지 이중화 구조

MIT·OFL은 "배포물에 저작권 표시와 라이선스 사본을 포함"할 것을 조건으로 건다. 웹 게임에서 이 의무를 확실하게 지키기 위해 두 경로를 모두 둔다.

```
src/data/credits.ts
├─ CreditsPanel
│   └─ 게임 내 「 제작자 」 화면 - 플레이 중 언제든지 열람
└─ public/THIRD-PARTY-NOTICES.md (같은 목록의 전문)
    빌드 시 dist/ 루트로 복사 → URL로 직접 열람 가능
```

새 라이브러리나 에셋을 추가하면 **양쪽 모두**에 항목을 추가하는 것이 규칙이다. 미확정 항목 수는 `pendingCount()`로 집계되어 크레딧 화면 상단 경고 배지에 표시된다.

7.5 외부 연동 서비스 (개발 전용)

서비스	용도	인증	배포물 포함 여부
Notion API	GDD · 개발 태스크 동기화	<code>NOTION_TOKEN</code> (<code>.env</code> , gitignore)	미포함
GitHub API (<code>gh</code> CLI)	이슈 조회 · PR 생성	로컬 <code>gh</code> 인증	미포함
Higgsfield MCP	컷씬 영상 생성(Seedance 2.0 호출)	서비스 계정 (저장소 외부)	미포함
PixelLab.ai	스프라이트 · 애니메이션 생성	서비스 계정 (저장소 외부)	미포함 (생성물만 커밋)
Suno AI	BGM 생성	서비스 계정 (저장소 외부)	미포함 (생성물만 커밋)
ElevenLabs	후원 효과음 생성	서비스 계정 (저장소 외부)	미포함 (생성물만 커밋)
GitHub Actions	<code>main</code> push 시 Pages 자동 배포	<code>secrets.GITHUB_TOKEN</code>	해당 없음
오브젝트 스토리지	컷씬 영상(<code>.mp4</code>) 호스팅	공개 읽기	URL만 포함

8. 부록 — 관련 파일 색인

파일	내용
<code>.claude/commands/</code>	슬래시 커맨드 5종 (스킬 진입점)
<code>.claude/skills/</code>	AI 지시문 본체 5종
<code>.claude/skills/notion/scripts/sync.mjs</code>	Notion API CLI (397줄)
<code>scripts/pack.js</code>	AI 생성 스프라이트 → Phaser 아틀라스 패커
<code>assets/character/</code>	캐릭터별 AI 생성 프롬프트·파라미터 원본(<code>metadata.json</code>)
<code>src/data/credits.ts</code>	크레딧·서드파티 고지 단일 출처
<code>public/THIRD-PARTY-NOTICES.md</code>	라이선스 전문 (배포물 동봉)
<code>CONTRIBUTING.md</code>	브랜치 전략 · 스킬 사용법 · 환경 변수
<code>GAME_INTRODUCTION.md</code>	게임 소개 및 설명 문서

9. 사용한 AI 도구 총괄

영역	도구	산출물
코드 구현 · 리팩터링 · 문서 동기화	Claude Code (모델 Opus 5)	소스 코드 · 테스트 · GDD/태스크 동기화
캐릭터 디스크립션 프롬프트 작성	Claude	metadata.json 의 생성 프롬프트
스프라이트 · 애니메이션	PixelLab.ai (image to pixel art + character)	public/assets/character/*.png + .json
컷씬 마스터 시트	Higgsfield	캐릭터 · 배경 기준 이미지
컷씬 스토리보드	GPT Image 2	컷 구성 이미지
컷씬 영상	Seedance 2.0 (Higgsfield MCP 경유)	/cutscenes/*.mp4
BGM	Suno AI	public/assets/sounds/bgm/
후원 효과음	ElevenLabs	public/assets/sounds/sfx/*_donation.mp3